

The Complete Git Roadmap

From Zero to Advanced | CLI Focused | Free Resources Only

Prepared by Jatin | [instagram.com/heyjatinnn](https://www.instagram.com/heyjatinnn)

A step-by-step learning path with every command explained, curated free resources (books, videos, cheatsheets, games), and real-world scenarios to practice along the way.

1

Foundation

Install, configure, first commits

2

Core Workflow

Add, commit, status, diff, log

3

Branching

Create, switch, merge, delete

4

Remote Repos

Clone, push, pull, fetch

5

Undo & Recovery

Reset, revert, reflog, restore

6

Advanced

Rebase, cherry-pick, bisect, worktree

7

Pro Workflows

Stash, tags, blame, aliases, hooks

Total estimated time: 4-6 weeks (1-2 hours/day)

Essential Resources (All Free)

Bookmark these before you start. You will reference them throughout your journey.

Books

BOOK

Pro Git (2nd Edition) by Scott Chacon & Ben Straub

<https://git-scm.com/book/en/v2>

The definitive Git book. Freely available online under Creative Commons license. Covers everything from basics to internals. Read chapters alongside each stage of this roadmap.

BOOK

Git Internals PDF (Pluralsight/GitHub)

<https://github.com/pluralsight/git-internals-pdf>

Deep dive into how Git works under the hood. Best read after Stage 4.

Cheatsheets (PDF)

CHEATSHEET

GitHub Education Git Cheat Sheet

<https://education.github.com/git-cheat-sheet-education.pdf>

CHEATSHEET

GitLab Git Cheat Sheet

<https://about.gitlab.com/images/press/git-cheat-sheet.pdf>

CHEATSHEET

Atlassian Git Cheat Sheet

<https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet>

CHEATSHEET

Wizard Zines Git Cheat Sheet (Visual)

<https://wizardzines.com/git-cheat-sheet.pdf>

Interactive Games & Practice

GAME

Learn Git Branching (Interactive Visual Tutorial)

<https://learngitbranching.js.org>

The single best interactive tool for understanding branching, merging, rebasing visually. Complete all levels.

GAME

Oh My Git! (Desktop Game for Beginners)

<https://ohmygit.org>

GAME

Game4Git (Browser-based Git Game)

<https://game4git.games>

Tutorials & Documentation

DOCS

Official Git Documentation

<https://git-scm.com/docs>

TUTORIAL

Atlassian Git Tutorials (Excellent Explanations)

<https://www.atlassian.com/git/tutorials>

TUTORIAL

W3Schools Git Tutorial (Beginner Friendly)

<https://www.w3schools.com/git>

TUTORIAL

The Odin Project - Git Basics (Free Curriculum)

<https://www.theodinproject.com/lessons/foundations-introduction-to-...>

Video Courses (Free)

VIDEO

Git & GitHub Crash Course - Traversy Media (YouTube)

https://www.youtube.com/watch?v=SWYqp7iY_Tc

VIDEO

Git Tutorial for Beginners - Net Ninja (YouTube Playlist)

https://www.youtube.com/playlist?list=PL4cUxeGkcC9goXbgTDQ0n_4TBzOO...

VIDEO

Git for Professionals - Fireship (YouTube)

https://www.youtube.com/watch?v=Uszj_k0DGsg

VIDEO

Git & GitHub - JavaScript Mastery (YouTube, 2024)

<https://www.youtube.com/watch?v=S7XpTAnSDL4>

TUTORIAL

Codecademy - Learn Git (Free Interactive Course)

<https://www.codecademy.com/learn/learn-git>

1

Foundation

Install Git, understand version control, make your first commit | Days 1-3

What is Git and Why Does It Exist?

Git is a distributed version control system created by Linus Torvalds in 2005. It tracks changes in your code over time, lets you go back to any previous state, and allows multiple people to work on the same project without overwriting each other's work. Before Git, teams used centralized systems like SVN where a single server held all the history. Git changed this by giving every developer a full copy of the entire project history.

Step 1: Install Git

```
$ sudo apt-get install git
```

Linux (Debian/Ubuntu)

```
$ brew install git
```

macOS (with Homebrew)

Windows: Download installer from <https://git-scm.com/download/win>

```
$ git --version
```

Verify installation

Step 2: Configure Git (One-Time Setup)

```
$ git config --global user.name "Your Name"
```

Set your name for all commits

```
$ git config --global user.email "you@email.com"
```

Set your email for all commits

```
$ git config --global init.defaultBranch main
```

Set default branch name to main

```
$ git config --list
```

View all your settings

Step 3: Understand the 3 Areas of Git

Git has three main areas you must understand before writing any command:

Working Directory - Your actual files on disk. This is where you edit code.

Staging Area (Index) - A preparation zone. Files here are ready to be committed.

Repository (.git) - Where Git permanently stores your committed snapshots.

The flow is always: Edit files -> Stage them (git add) -> Commit them (git commit).

Step 4: Your First Repository

```
$ mkdir my-project && cd my-project
```

Create a project folder

```
$ git init
```

Initialize a Git repo (creates .git folder)

```
$ echo 'Hello Git' > readme.txt
```

Create your first file

```
$ git add readme.txt
```

Stage the file (move to staging area)

```
$ git commit -m "Initial commit"
```

Commit the snapshot

PRO TIP:

Think of commits as save points in a video game. Each commit is a snapshot you can return to at any time. Make small, frequent commits with clear messages.

Stage 1 Resources

BOOK

Pro Git Book - Chapter 1: Getting Started

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

BOOK

Pro Git Book - Chapter 2: Git Basics

<https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>

TUTORIAL

W3Schools - Git Getting Started

https://www.w3schools.com/git/git_getstarted.asp

Stage 1 Checklist

- ☐ Git installed and version verified
- ☐ Name and email configured globally
- ☐ Created first repository with git init
- ☐ Made first commit with a clear message
- ☐ Can explain Working Directory vs Staging Area vs Repository

2

Core Workflow

Master the daily commands every developer uses | Days 4-7

Step 1: Tracking Changes

\$ git status

Check what changed (your most-used command)

\$ git add <file>

Stage a specific file

\$ git add .

Stage ALL changed files at once

\$ git commit -m "descriptive message"

Commit staged changes

\$ git commit -am "message"

Stage + commit tracked files in one step

Step 2: Viewing History & Changes

\$ git log

See commit history (press q to exit)

\$ git log --oneline

Compact one-line view of history

\$ git log --oneline --graph --all

Visual branch graph in terminal

\$ git log -p -2

Show actual code changes for last 2 commits

\$ git log --stat

Summary of files changed per commit

\$ git diff

See unstaged changes (what you edited but haven't added)

\$ git diff --staged

See staged changes (what's ready to commit)

\$ git diff HEAD

See ALL changes (staged + unstaged)

Step 3: The .gitignore File

Create a .gitignore file in your project root to tell Git which files to ignore. This prevents accidentally committing build files, dependencies, or secrets.

\$ node_modules/

Example: ignore node_modules folder

\$ *.log

Example: ignore all .log files

\$.env

Example: ignore environment secrets

Good Commit Messages

Write messages that explain WHAT changed and WHY. Use present tense, be specific.

Good: "Add user authentication with JWT tokens"

Bad: "fixed stuff" or "update"

Format: type(scope): description. Examples: feat(auth): add login endpoint, fix(api): handle null response.

PRO TIP:

Run 'git status' before every commit. It becomes muscle memory. Also run 'git diff --staged' before committing to double-check what you're actually committing.

Stage 2 Resources

BOOK

Git Basics - Recording Changes (Pro Git Ch.2)

<https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-...>

TUTORIAL

gitignore.io - Generate .gitignore for any tech stack

<https://www.toptal.com/developers/gitignore>

Stage 2 Checklist

- ☐ Can check status, add files, and commit confidently
- ☐ Understand git diff vs git diff --staged
- ☐ Can read git log in multiple formats
- ☐ Created a proper .gitignore file
- ☐ Writing clear, descriptive commit messages

3

Branching & Merging

Work on features without breaking main code | Days 8-12

Why Branches Matter

A branch is a lightweight pointer to a commit. Branches let you work on features, bug fixes, or experiments in isolation without touching the main codebase. When your work is ready, you merge it back. This is the foundation of team collaboration.

Step 1: Creating & Switching Branches

```
$ git branch
```

List all local branches

```
$ git branch <name>
```

Create a new branch

```
$ git checkout <name>
```

Switch to a branch

```
$ git checkout -b <name>
```

Create AND switch in one command (use this!)

```
$ git switch <name>
```

Modern way to switch branches (Git 2.23+)

```
$ git switch -c <name>
```

Modern way to create + switch

```
$ git branch -vv
```

List branches with upstream info and last commit

Step 2: Merging Branches

To merge a feature branch into main: first switch to main, then merge the feature branch in.

```
$ git checkout main
```

Switch to the target branch

```
$ git merge feature-branch
```

Merge feature-branch INTO main

```
$ git branch -d feature-branch
```

Delete branch after merge (safe delete)


```
$ git branch -D feature-branch
```

Force delete (even if unmerged)

Step 3: Handling Merge Conflicts

Conflicts happen when two branches edit the same lines. Git marks the conflict in your file with <<<<<<, =====, and >>>>>> markers. To resolve: open the file, choose which code to keep, remove the markers, then add and commit.

```
$ git merge --abort
```

Cancel a merge if conflicts are too complex

Step 4: Visualize Your Branches

```
$ git log --oneline --decorate --graph --all
```

See full branch graph

PRO TIP:

Always create a new branch for every feature or bug fix. Never commit directly to main. This habit will save you in team environments.

Stage 3 Resources

BOOK

Pro Git Ch.3 - Git Branching

<https://git-scm.com/book/en/v2/Git-Branching-Branched-in-a-Nutshell>

GAME

Learn Git Branching (MUST DO - Complete All Levels)

<https://learngitbranching.js.org>

TUTORIAL

Atlassian - Using Branches

<https://www.atlassian.com/git/tutorials/using-branches>

Stage 3 Checklist

- ☐ Can create, switch, and delete branches
- ☐ Successfully merged a branch into main
- ☐ Resolved at least one merge conflict manually
- ☐ Completed Learn Git Branching main levels
- ☐ Understand the branch graph visualization

4

Working with Remotes

Push, pull, and collaborate with remote repositories | Days 13-17

What is a Remote?

A remote is a copy of your repository hosted on a server (like GitHub, GitLab, or Bitbucket). It allows you to share code, collaborate with others, and keep a backup. The default remote is usually called 'origin'.

Step 1: Clone an Existing Repo

```
$ git clone <url>
```

Download a repo and its entire history

```
$ git clone <url> <folder-name>
```

Clone into a specific folder name

Step 2: Connect a Local Repo to Remote

```
$ git remote add origin <url>
```

Link your local repo to a remote

```
$ git remote -v
```

View configured remotes

```
$ git remote remove origin
```

Remove a remote connection

Step 3: Push & Pull

```
$ git push -u origin main
```

Push main branch (first time, sets upstream)

```
$ git push
```

Push commits (after upstream is set)

```
$ git push origin <branch>
```

Push a specific branch

Step 4: Fetch vs Pull (Critical Difference)

This is one of the most important distinctions in Git:

```
$ git fetch origin
```

Download changes WITHOUT merging (safe)

\$ git log origin/main

Review what was fetched before merging

\$ git merge origin/main

Merge fetched changes when YOU decide

\$ git pull origin main

Fetch + merge in one step (risky if conflicts)

PRO TIP:

Use 'git fetch' + review + 'git merge' instead of 'git pull'. It gives you full control and prevents surprise merge conflicts from breaking your code.

Step 5: SSH Setup (Stop Typing Passwords)

\$ ssh-keygen -t ed25519 -C "your@email.com"

Generate SSH key

\$ cat ~/.ssh/id_ed25519.pub

Copy this key to GitHub/GitLab settings

Stage 4 Resources

BOOK

Pro Git Ch.2.5 - Working with Remotes

<https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes>

GAME

Learn Git Branching - Remote Levels (Complete These!)

https://learngitbranching.js.org/?locale=en_US

DOCS

GitHub Docs - Connecting with SSH

<https://docs.github.com/en/authentication/connecting-to-github-with...>

Stage 4 Checklist

- ☐ Cloned a repository from GitHub
- ☐ Pushed local commits to a remote repo
- ☐ Understand fetch vs pull and when to use each
- ☐ Can view and manage remote connections
- ☐ Set up SSH keys for passwordless push

5

Undo & Recovery

Fix mistakes, recover lost work, travel through time | Days 18-22

This is where Git goes from useful to life-saving. Every command here exists because developers make mistakes. Knowing these commands means you never permanently lose work.

Step 1: Unstage Files

```
$ git restore --staged <file>
```

Remove file from staging (keep changes)

```
$ git restore <file>
```

Discard changes in working directory (DESTRUCTIVE)

Step 2: Amend Last Commit

```
$ git commit --amend -m 'new message'
```

Fix the last commit message

```
$ git commit --amend --no-edit
```

Add forgotten files to last commit

Step 3: Git Reset (Move HEAD Backward)

```
$ git reset --soft HEAD~1
```

Undo last commit, keep changes staged

```
$ git reset --mixed HEAD~1
```

Undo last commit, keep changes unstaged (default)

```
$ git reset --hard HEAD~1
```

Undo last commit AND delete all changes (DANGEROUS)

PRO TIP:

Rule of thumb: if you're not sure, never use --hard. Use --soft to keep your code safe while undoing the commit.

Step 4: Git Revert (Safe Undo for Shared History)

Unlike reset which rewrites history, revert creates a NEW commit that undoes a previous one. Use revert on shared/pushed branches. Use reset only on local unpushed commits.

```
$ git revert <commit-id>
```

Create a new commit that undoes the target commit

```
$ git revert HEAD
```

Revert the most recent commit

Step 5: Git Reflog (Your Safety Net)

Reflog records every movement of HEAD, even after reset or branch deletion. It is Git's secret diary. If you think you lost work, reflog probably has it.

\$ git reflog

View the full history of HEAD movements

\$ git checkout <commit-id>

Jump to a specific state from reflog

\$ git reset --hard <commit-id>

Restore to a reflog entry permanently

Recovery Scenarios

Accidentally deleted a branch? Run git reflog, find the commit, run git branch <name> <commit-id>.

Used reset --hard and lost code? Run git reflog, find the commit before reset, run git reset --hard <commit-id>.

Need to undo a pushed commit? Use git revert, never use git reset on pushed commits.

Stage 5 Resources

BOOK

Pro Git Ch.2.4 - Undoing Things

<https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things>

TUTORIAL

Atlassian - Resetting, Checking Out, and Reverting

<https://www.atlassian.com/git/tutorials/resetting-checking-out-and-...>

TUTORIAL

Oh Shit, Git!?! (Emergency Recovery Guide)

<https://ohshitgit.com>

Stage 5 Checklist

- ☐ Can unstage files with git restore --staged
- ☐ Amended a commit message and added forgotten files
- ☐ Understand soft vs mixed vs hard reset
- ☐ Used git revert on a pushed commit
- ☐ Recovered lost work using git reflog

6

Advanced Git

Rebase, cherry-pick, bisect, worktree - the power commands | Days 23-30

Step 1: Rebase (Clean Linear History)

Rebase moves your branch's commits on top of another branch, creating a straight-line history instead of merge commits. It rewrites history, so use it **ONLY** on local/feature branches, never on shared branches like main.

```
$ git rebase main
```

Rebase current branch onto main

```
$ git rebase -i HEAD~3
```

Interactive rebase: edit/squash last 3 commits

```
$ git rebase --abort
```

Cancel rebase if conflicts are messy

PRO TIP:

Golden rule: Feature branch -> rebase. Shared branch like main -> merge. Rebase rewrites history, which breaks things for teammates if done on shared branches.

Step 2: Cherry-Pick (Steal a Single Commit)

Cherry-pick copies a specific commit from one branch and applies it to your current branch. Useful for applying a bug fix from a feature branch to main without merging everything.

```
$ git cherry-pick <commit-id>
```

Apply one commit to current branch

```
$ git cherry-pick -n <commit-id>
```

Apply changes without committing (stage only)

Step 3: Bisect (Find the Bug with Binary Search)

When a bug exists but you don't know which commit caused it, bisect does a binary search through your commit history. You tell it which commits are good/bad, and it narrows down the culprit in logarithmic time (200 commits = only ~8 checks).

```
$ git bisect start
```

Start the bisect session

```
$ git bisect bad
```

Mark current commit as bad (has bug)

```
$ git bisect good <commit-id>
```

Mark a known working commit

\$ git bisect good / git bisect bad

Repeat until Git finds the culprit

\$ git bisect reset

End the bisect session

Step 4: Worktree (Multiple Branches Simultaneously)

Worktree lets you check out a different branch in a separate folder, so you can work on two branches at the same time without stashing or switching.

\$ git worktree add ../hotfix main

Create new folder with main branch

\$ git worktree list

See all active worktrees

\$ git worktree remove ../hotfix

Clean up when done

Step 5: Checkout Branch from Old Commit

\$ git checkout -b new-branch <commit-hash>

Create branch from a specific past commit

Stage 6 Resources

BOOK

Pro Git Ch.3.6 - Rebasing

<https://git-scm.com/book/en/v2/Git-Branching-Rebasing>

TUTORIAL

Atlassian - Merging vs Rebasing

<https://www.atlassian.com/git/tutorials/merging-vs-rebasing>

GAME

Learn Git Branching - Advanced Levels

<https://learngitbranching.js.org>

DOCS

Git Docs - git-bisect

<https://git-scm.com/docs/git-bisect>

Stage 6 Checklist

- ☐ Rebased a feature branch onto main
- ☐ Used interactive rebase to squash commits
- ☐ Cherry-picked a commit between branches
- ☐ Found a bug using git bisect
- ☐ Used git worktree to work on two branches simultaneously

7

Pro Workflows & Productivity

Stash, tags, blame, aliases, hooks, and daily efficiency | Days 31-40+

Step 1: Git Stash (Save Work Temporarily)

\$ git stash

Save uncommitted changes and clean working directory

\$ git stash save "work in progress on login"

Stash with a descriptive message

\$ git stash list

View all saved stashes

\$ git stash pop

Apply most recent stash and remove it from list

\$ git stash apply

Apply stash but keep it in the list

\$ git stash drop stash@{0}

Delete a specific stash

\$ git stash clear

Remove all stashes

Step 2: Git Tags (Mark Releases)

\$ git tag v1.0.0

Create a lightweight tag

\$ git tag -a v1.0.0 -m "First release"

Create an annotated tag (recommended)

\$ git tag

List all tags

\$ git push origin --tags

Push all tags to remote

Step 3: Git Blame (Who Wrote This Line?)

\$ git blame <file>

Show who last modified each line and when


```
$ git blame -L 10,20 <file>
```

Blame only lines 10-20

```
$ git log -p <file>
```

Full change history of a specific file

```
$ git shortlog -sn
```

Contribution count per author (for sprint reviews)

Step 4: Git Aliases (Type Less, Do More)

```
$ git config --global alias.co checkout
```

Now 'git co' = 'git checkout'

```
$ git config --global alias.br branch
```

Now 'git br' = 'git branch'

```
$ git config --global alias.st status
```

Now 'git st' = 'git status'

```
$ git config --global alias.lg 'log --oneline --graph --all'
```

Pretty graph shortcut

Step 5: Git Clean (Remove Untracked Files)

```
$ git clean -n
```

Preview what would be deleted (dry run)

```
$ git clean -fd
```

Delete untracked files and directories

Stage 7 Resources

BOOK

Pro Git Ch.7 - Git Tools (Stashing, Searching, etc.)

<https://git-scm.com/book/en/v2/Git-Tools-Stashing-and-Cleaning>

BOOK

Pro Git Ch.2.6 - Tagging

<https://git-scm.com/book/en/v2/Git-Basics-Tagging>

DOCS

Git Docs - Customizing Git (Aliases & Hooks)

<https://git-scm.com/book/en/v2/Customizing-Git-Git-Configuration>

Stage 7 Checklist

- ☐ Used git stash to save and restore work
- ☐ Created tags for a release version
- ☐ Used git blame to find who changed a line

- ☐ Set up at least 3 useful aliases
- ☐ Used git clean to remove untracked files

Quick Reference: All Commands by Category

Setup & Config

<code>git init</code>	Initialize new repo
<code>git clone <url></code>	Clone remote repo
<code>git config --global user.name</code>	Set name
<code>git config --global user.email</code>	Set email

Daily Commands

<code>git status</code>	Check state
<code>git add <file></code>	Stage file
<code>git add .</code>	Stage all
<code>git commit -m 'msg'</code>	Commit
<code>git diff</code>	View changes
<code>git log --oneline</code>	View history

Branching

<code>git branch <name></code>	Create branch
<code>git checkout -b <name></code>	Create + switch
<code>git merge <branch></code>	Merge branch
<code>git branch -d <name></code>	Delete branch

Remote

<code>git push</code>	Push to remote
<code>git fetch</code>	Download (safe)
<code>git pull</code>	Fetch + merge
<code>git remote -v</code>	View remotes

Undo

<code>git restore --staged <f></code>	Unstage
<code>git reset --soft HEAD~1</code>	Undo commit (keep)
<code>git reset --hard HEAD~1</code>	Undo commit (delete)
<code>git revert <id></code>	Safe undo
<code>git reflog</code>	Recovery diary

Advanced

<code>git rebase main</code>	Rebase onto main
<code>git cherry-pick <id></code>	Copy one commit
<code>git bisect start</code>	Binary bug search
<code>git worktree add</code>	Multi-branch work
<code>git stash</code>	Save temp work
<code>git blame <file></code>	Line-by-line author

*Keep this roadmap. Follow it step by step. Practice every command in a real repository.
The best way to learn Git is to use Git. Start today.*